

Práctica 2: Búsquedas con Costes y Traza

Universidad de Huelva Inteligencia Artificial - Curso 2024-2025

Introducción

El objetivo de esta práctica es que los estudiantes implementen algoritmos de búsqueda fundamentales y profundicen en su funcionamiento interno. A diferencia de un enfoque de "caja negra", en esta práctica se requiere visualizar **cómo evoluciona la frontera de búsqueda (lista de ABIERTOS)** y la memoria del agente (lista de CERRADOS) paso a paso. Además, se introduce un entorno de **costes asimétricos**, donde moverse en ciertas direcciones es más costoso, lo que afectará la decisión de los algoritmos de búsqueda informada y de coste uniforme.

Problema

Partiendo de la estructura de la práctica anterior, se deben implementar algoritmos de búsqueda para resolver laberintos definidos por paredes (#), una entrada (E) y una salida (S).

Nuevas Reglas de Movimiento y Costes

El entorno ya no es de coste uniforme. El agente gasta más energía al moverse "en contra" de la gravedad o retroceder (metafóricamente). Los costes son:

- **Mover ARRIBA:** Coste 2.
- **Mover IZQUIERDA:** Coste 2.
- **Mover ABAJO:** Coste 1.
- **Mover DERECHA:** Coste 1.

Salida del Programa (Modo Verbose/Traza)

El programa debe tener un modo de ejecución que muestre paso a paso la evolución de las estructuras de datos.

Ejemplo de Salida Esperada (Algoritmo A):*

```
Ciclo 0 ---
#####
#A   #
#   #
#   #S#
#####

Iniciando búsqueda A*...
Origen: (1,1) -> Meta: (3,5)

[PASO 1]
Seleccionado: (1,1) [f=6, g=0, h=6]
Generados hijos: (2,1), (1,2)
Estado ABIERTOS: [(2,1), (1,2)]
Estado CERRADOS: [(1,1)]

[PASO 2]
Seleccionado: (2,1) [f=6, g=1, h=5]
Generados hijos: (3,1)
Estado ABIERTOS: [(1,2), (3,1)]
Estado CERRADOS: [(2,1), (1,1)]

[PASO 3]
Seleccionado: (1,2) [f=6, g=1, h=5]
Generados hijos: (1,3)
Estado ABIERTOS: [(3,1), (1,3)]
Estado CERRADOS: [(2,1), (1,1), (1,2)]
```

```

[PASO 4]
Seleccionado: (3,1) [f=6, g=2, h=4]
Generados hijos: (3,2)
Estado ABIERTOS: [(1,3), (3,2)]
Estado CERRADOS: [(2,1), (1,1), (1,2), (3,1)]

[PASO 5]
Seleccionado: (1,3) [f=6, g=2, h=4]
Generados hijos: (1,4)
Estado ABIERTOS: [(3,2), (1,4)]
Estado CERRADOS: [(2,1), (1,1), (1,2), (1,3), (3,1)]

[PASO 6]
Seleccionado: (3,2) [f=6, g=3, h=3]
Generados hijos: (3,3)
Estado ABIERTOS: [(1,4), (3,3)]
Estado CERRADOS: [(2,1), (3,2), (1,1), (1,2), (1,3), (3,1)]

[PASO 7]
Seleccionado: (1,4) [f=6, g=3, h=3]
Generados hijos: (1,5)
Estado ABIERTOS: [(3,3), (1,5)]
Estado CERRADOS: [(2,1), (3,2), (1,1), (1,2), (1,3), (1,4), (3,1)]

[PASO 8]
Seleccionado: (3,3) [f=6, g=4, h=2]
Generados hijos: Ninguno
Estado ABIERTOS: [(1,5)]
Estado CERRADOS: [(2,1), (3,2), (1,1), (3,3), (1,2), (1,3), (1,4), (3,1)]

[PASO 9]
Seleccionado: (1,5) [f=6, g=4, h=2]
Generados hijos: (2,5)
Estado ABIERTOS: [(2,5)]
Estado CERRADOS: [(2,1), (3,2), (1,1), (3,3), (1,2), (1,3), (1,4), (1,5), (3,1)]

[PASO 10]
Seleccionado: (2,5) [f=6, g=5, h=1]
Generados hijos: (3,5)
Estado ABIERTOS: [(3,5)]
Estado CERRADOS: [(2,1), (3,2), (1,1), (3,3), (1,2), (1,3), (1,4), (2,5), (1,5), (3,1)]

--- ESTADÍSTICAS FINALES ---
Solución encontrada usando algoritmo A*.
Camino recorrido (x,y): [[1,1], [1,2], [1,3], [1,4], [1,5], [2,5], [3,5]].
Coste total del camino: 6.
Nodos expandidos: 11.
Tamaño máximo de ABIERTOS: 2.
Tiempo de ejecución: 1,45 ms.
A
gente decide moverse hacia: E

--- Ciclo 1 ---
#####
#.A #
# # # #
# #S#
#####
Agente decide moverse hacia: E
--- Ciclo 2 ---
#####
#..A #
# # # #
# #S#
#####
Agente decide moverse hacia: E
--- Ciclo 3 ---
#####
#...A #
# # # #
# #S#
#####
Agente decide moverse hacia: E
--- Ciclo 4 ---
#####
#...A#
# # # #
# #S#
#####

```

```

Agente decide moverse hacia: S
--- Ciclo 5 ---
#####
#.....#
#  ###A#
#    #S#
#####
Agente decide moverse hacia: S
>>> ¡VICTORIA! Meta alcanzada.
#####
#.....#
#  ###.#
#    #S#
#####

```

Nota visual:

- **ABIERTOS (Frontera):** Debe mostrar los nodos ordenados según la estrategia del algoritmo (Cola para Anchura, Pila para Profundidad, Cola de Prioridad por $f(n)$ para A*).
- **CERRADOS:** Lista de nodos ya expandidos.

Requisitos Funcionales

1. **Implementar los siguientes algoritmos de búsqueda:**
 - **Búsqueda en Anchura (Breadth-First Search - BFS):** Implementada con una Cola (FIFO).
 - **Búsqueda en Profundidad (Depth-First Search - DFS):** Implementada con una Pila (LIFO).
 - **Búsqueda A* (A-Star):** Implementada con una Cola de Prioridad ordenada por $f(n)=g(n)+h(n)$.
 2. **Gestión de Costes:**
 - Todos los algoritmos deben calcular el coste acumulado $g(n)$ correctamente según las reglas (Arriba/Izq = 2, Abajo/Der = 1).
 - *Nota importante:* Debes analizar si Búsqueda en Anchura sigue garantizando la solución óptima (menor coste) con estos costes variables.
 3. **Visualización (Traza):**
 - La traza debe mostrar claramente qué nodo se extrae de **ABIERTOS** y cómo quedan las listas tras la expansión.
 - *Avanzado :* Mostrar y explicar tratamiento de repetidos cuando ocurre
 4. **Heurísticas para A*:**
 - Implementar Distancia Manhattan.
 - Implementar una heurística propia propuesta por el alumno que intente compensar los costes asimétricos (ej. penalizar más la distancia vertical si se va hacia arriba).
-

Estructura del Informe

1. **Introducción:** Objetivos y descripción del problema de costes asimétricos.
2. **Diseño de Algoritmos:**
 - Explicación de las estructuras de datos usadas (Stack, Queue, PriorityQueue).
 - Cómo se ha implementado el cálculo de costes.
 - Capturas de pantalla de la "Trazas" mostrando la diferencia de elección entre Profundidad (se hunde rápido en el laberinto) y Anchura (crece por capas).
3. **Análisis de Resultados (Crucial):**
 - Comparativa de costes finales y nodos expandidos. Rellenar tabla

Algoritmo	Coste Final	Nodos Expandidos	Tamaño Máx. ABIERTOS
Anchura (BFS)	(Rellenar)	(Rellenar)	(Rellenar)
Profundidad (DFS)	(Rellenar)	(Rellenar)	(Rellenar)
A* (Manhattan)	(Rellenar)	(Rellenar)	(Rellenar)

- **Análisis de Optimalidad:** ¿Encuentra el algoritmo de Anchura el camino de menor coste en este escenario? ¿Por qué? Muestra ejecuciones concretas.
 - **Análisis de A*:** Comportamiento de la heurística con los nuevos costes. Manhattan coste uniforme y teniendo en cuenta los costes personalizados.
4. **Conclusiones:** Discusión sobre qué estrategia es mejor para este tipo de movimiento "pesado" hacia arriba/izquierda.

Planificación Estimada

- **Semana 1:** Adaptación de la clase `Nodo` para incluir costes variables, acción y padres. Implementación de la visualización de listas (toString de colas). Algoritmos Anchura y Profundidad.
- **Semana 2:** Implementación de A* con cola de prioridad y gestión de costes $g(n)$ y $f(n)$. Implementación de heurísticas.
- **Semana 3:** Ejecución de experimentos, captura de trazas significativas y redacción del análisis comparativo.

Rúbrica de Evaluación

Calificación Criterios Específicos

Suspense	Los algoritmos no compilan o no resuelven el laberinto. No se implementan los costes variables (Arriba/Izq = 2). No existe la visualización de Abiertos/Cerrados. El informe es inexistente o copia.
Aprobado	Los 3 algoritmos (Anchura, Profundidad, A*) funcionan y encuentran camino. Se aplican los costes correctamente. Existe un modo "verbose" que muestra la traza básica. El informe responde a las preguntas básicas.
Notable	La traza es detallada y clara (muestra valores f , g , h). Se muestra tratamiento de repetidos. El análisis discute correctamente por qué Anchura pierde la propiedad de optimalidad de coste en grafos ponderados (o si usa Coste Uniforme). Se comparan las heurísticas. Código bien estructurado.
Sobresaliente	Implementación impecable y eficiente. Memoria con mapas específicos y ejemplos que justifican posibles mejoras.

Fecha de entrega : 17 de Abril

Guía de Integración: Búsquedas y Entorno

Para integrar la Práctica 2 en el bucle de simulación de la Práctica 1, necesitamos hacer un pequeño "cambio de paradigma". En la Práctica 1, el agente pensaba un solo paso por cada ciclo. En la Práctica 2, el agente debe pensar toda la ruta de golpe antes de moverse, y luego simplemente ejecutar ese plan paso a paso.

1. El Nuevo Agente: AgenteBusqueda

Este agente actuará como puente. Calculará la ruta completa la primera vez que se le pregunte y guardará los pasos en una cola. En los ciclos siguientes, simplemente irá sacando acciones de esa lista.

```
// =====
// NUEVO: AGENTE QUE PLANIFICA RUTAS
// =====
class AgenteBusqueda {
    private Queue<String> planDeAccion = new LinkedList<>();
    private boolean yaCalculado = false;

    public String pensar(Entorno entorno) {
        // Solo calculamos la ruta completa en el primer ciclo
        if (!yaCalculado) {
            System.out.println("Agente calculando ruta...");
            BusquedaAnchura buscador = new BusquedaAnchura();
            List<String> camino = buscador.resolver(entorno);

            if (camino != null) {
                planDeAccion.addAll(camino);
            } else {
                System.out.println("No hay ruta.");
            }
            yaCalculado = true;
        }

        // Ejecutamos el siguiente paso del plan
        if (!planDeAccion.isEmpty()) {
            return planDeAccion.poll();
        }
        return null;
    }
}
```

Actualizad el método main para utilizar el nuevo AgenteBusqueda. Fijaos en que el bucle while sigue siendo el mismo, pero ahora el agente no "improvisa" en cada ciclo, sino que sigue un plan.

```
public static void main(String[] args) throws InterruptedException {
    String mapaTexto =
        "#####\n" +
        "#E      #\n" +
        "#   ## #\n" +
        "#      #S#\n" +
        "#####";

    Entorno juego = new Entorno(mapaTexto);
    AgenteBusqueda agente = new AgenteBusqueda(); // Nuestro nuevo agente

    System.out.println("--- Inicio de la Simulación ---");
    boolean juegoActivo = true;
    int ciclos = 0;

    while (juegoActivo && ciclos < 100) {
        System.out.println("\n\n--- Ciclo " + ciclos + " ---");
        juego.dibujar();

        String accion = agente.pensar(juego);

        if (accion == null) {
            System.out.println("El agente se ha quedado sin plan o no encontró ruta.");
            break;
        }
        ...
    }
}
```

2. Nodo

Para realizar una búsqueda, el agente necesita recordar no solo dónde está, sino cómo llegó ahí y cuánto le costó. Para eso sirve la clase `Nodo`.

¿Por qué usar `Comparable`? Cuando implementéis `A*`, usaréis una `PriorityQueue`. Esta cola ordena automáticamente, pero debemos enseñarle a Java las reglas: "Por favor, cuando ordenes estos nodos, pon primero los que tengan un valor `f` más pequeño". Eso se hace sobrescribiendo el método `compareTo`.

```
class Nodo implements Comparable<Nodo> {
    int f, c;
    Nodo padre;
    String accion;
    int g;
    int h;
    int valorF;

    public Nodo(int f, int c, Nodo padre, String accion, int g, int h) {
        this.f = f;
        this.c = c;
        this.padre = padre;
        this.accion = accion;
        this.g = g;
        this.h = h;
        this.valorF = g + h;
    }

    @Override
    public int compareTo(Nodo otro) {
        return Integer.compare(this.valorF, otro.valorF);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Nodo nodo = (Nodo) o;
        return f == nodo.f && c == nodo.c;
    }

    @Override
    public int hashCode() {
        return Objects.hash(f, c);
    }

    @Override
    public String toString() {
        return "(" + f + ", " + c + ")";
    }
}
```

3. El Algoritmo y la Reconstrucción del Camino

Esquema del algoritmo y la función que retrocede por el árbol. Cuando el algoritmo encuentra la meta, tiene que "tirar del hilo" hacia atrás usando el atributo padre hasta llegar al inicio.

```
class BusquedaXX {
    public List<String> resolver(Entorno mapa) {
        // TODO: Lógica de inicialización, bucles, CERRADOS...
        return null;
    }

    private List<String> reconstruirCamino(Nodo meta) {
        LinkedList<String> camino = new LinkedList<>();
        Nodo actual = meta;

        while (actual.padre != null) {
            camino.addFirst(actual.accion);
            actual = actual.padre;
        }
        System.out.println("Coste total: " + meta.g);
        return camino;
    }
}
```

4. Clase entorno

1. El Constructor ahora entiende la "E" y la "A": En la Práctica 1 el agente se representaba con la letra "A", pero en el mapa de texto de la Práctica 2 la entrada se marca con una "E". El constructor se ha modificado para que, si lee una "E" o una "A", guarde las coordenadas iniciales (agenteF, agenteC) y lo dibuje unificadamente como una "A".

Método nuevo - esMeta(int f, int c): En la P1 se usaba hemosGanado() para comprobar si la posición actual del agente coincidía con la meta. Sin embargo, los algoritmos de búsqueda (A*, BFS, DFS) exploran nodos virtuales antes de moverse. Por eso se añade esMeta(f, c), permitiendo al algoritmo preguntar si una coordenada específica es el destino.

```
class Entorno {
    char[][] grid;
    int filas, cols;
    int agenteF, agenteC;
    int metaF, metaC;

    public Entorno(String mapa) {
        String[] lineas = mapa.trim().split("\n");
        filas = lineas.length;
        cols = lineas[0].length();
        grid = new char[filas][cols];

        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < cols; j++) {
                char celda = lineas[i].charAt(j);
                grid[i][j] = celda;
            }
        }
        // MODIFICACIÓN 1: Tratamos 'E' y 'A' como el inicio del agente
        if (celda == 'E' || celda == 'A') { agenteF = i; agenteC = j; grid[i][j] = 'A'; }
        if (celda == 'S') { metaF = i; metaC = j; }
    }
}
```

```
// MODIFICACIÓN 2: Nuevo método para evaluar nodos virtuales (usado por A*, BFS, DFS)
public boolean esMeta(int f, int c) {
    return f == metaF && c == metaC;
}

public boolean moverAgente(String accion) {
    int nf = agenteF, nc = agenteC;
    if (accion.equalsIgnoreCase("N")) nf--;
    if (accion.equalsIgnoreCase("S")) nf++;
    if (accion.equalsIgnoreCase("E")) nc++;
    if (accion.equalsIgnoreCase("O")) nc--;

    if (!esTransitable(nf, nc)) return false;

    grid[agenteF][agenteC] = '.'; // Deja rastro por donde pasa
    agenteF = nf; agenteC = nc;

    if (grid[nf][nc] != 'S') grid[nf][nc] = 'A'; // Pinta al agente
    return true;
}

public boolean hemosGanado() {
    return agenteF == metaF && agenteC == metaC;
}

public void dibujar() {
    for (char[] fila : grid) {
        System.out.println(new String(fila));
    }
}
}
```